

Machine Learning-Driven STL Generation for Enhancing Functional Safety of E/E Systems

Sanjay Das^{1*}, Swastik Bhattacharya¹, Anand Menon¹, Shamik Kundu³, Pooja Madhusoodhanan², Prasanth Viswanathan Pillai², Rubin Parekhji², Arnab Raha³, Suvadeep Banerjee³, Suriyaprakash Natarajan⁴ and Kanad Basu¹

¹University of Texas at Dallas, TX, USA; ²Texas Instruments, Bangalore, India; ³Intel Corporation, CA, USA;

⁴Siemens EDA, OR, USA

Abstract—The increasing complexity of safety-critical hardware systems demands advanced methods for ensuring functional safety (FuSa). Traditional techniques like ATPG and BIST are intrusive, requiring additional hardware and disrupting operations, making them unsuitable for in-field testing. To address this, for the first time, we propose a machine learning (ML)-driven automated Self-Test Library (STL) generation for seamless in-field testing during idle periods, ensuring uninterrupted fault detection and high system performance. Utilizing reinforcement learning, the STL generates design-specific test patterns, achieving up to 57.57% improvement in fault coverage and up to 85% efficiency compared to existing pattern-based testing, enhancing FuSa in mission-critical applications.

Index Terms—Functional Safety, Fault Injection Analysis, Test Pattern Generation, Self-Test Library Development.

I. INTRODUCTION

With increasing design complexity, shrinking feature sizes, and accelerated time-to-market requirements, hardware systems are exposed to a broad spectrum of potential failures, primarily due to hardware faults. This has led to the development of Functional Safety (FuSa) standards, such as ISO 26262, which provide a technical framework for detecting and managing faults, ensuring safe operational states [1]. Faults in critical circuits can have severe consequences during operation, with the potential to cause catastrophic failures [2]. For instance, a fault in the control system of an autonomous vehicle could lead to uncontrollable acceleration, resulting in accidents or even loss of life, as shown in Figure 1. The severity of these faults depends on their location within the circuit and the structural characteristics of the design. Fault propagation is closely tied to these structural features, making it crucial to assess both the impact and criticality of faults in relation to their location within the design. This comprehensive approach is imperative for ensuring FuSa in mission-critical applications. A paradigm shift in safety assurance is anticipated with the prospect of machine learning (ML)-driven automation in FuSa. This is set to strengthen the strictness of fault management procedures in sectors that depend on risky, safety-critical hardware systems. With ramifications for the future of autonomous systems, the Internet of Things (IoT), and other mission-critical sectors, this research represents a crucial frontier. In addition to being a tempting opportunity, the ongoing investigation of ML for automated functional safety is also essential to stay up with the complexity of next-generation hardware.

The increasing complexity of modern E/E systems has rendered traditional fault injection and simulation techniques inadequate for ensuring FuSa. These methods face substantial timing overheads, are resource-intensive, and often fail to identify critical design components that could lead to functional failures. Furthermore, they lack

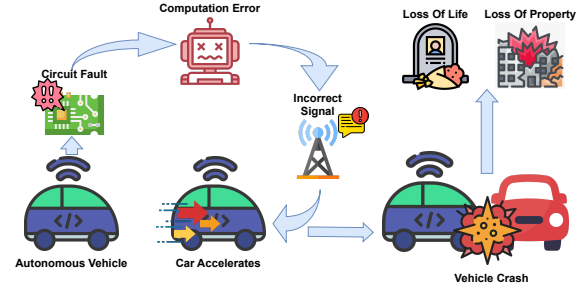


Fig. 1. Detrimental effects of fault manifestations in E/E systems.

the capability to quantify the varying degrees of criticality among circuit nodes, which is essential for prioritizing fault mitigation efforts. Conventional fault analysis techniques, therefore, fall short of addressing the growing intricacy of contemporary systems. To overcome these challenges, ML-based approaches have emerged as a necessary solution for enhancing FuSa [3]–[5]. By efficiently generating high-quality test patterns, these methods not only improve fault coverage but also provide a more systematic framework for detecting critical faults within complex designs. The use of graph-based analysis, specifically, has proven effective in understanding the structural features of E/E systems, allowing for more precise fault criticality analysis [6].

The automotive functional safety standard ISO 26262 [1] and the industrial standard IEC 61508 [7] stipulate the necessity for evaluating both permanent and transient fault coverage during runtime operations. Attaining the requisite level of permanent fault coverage presents considerable challenges. There are three principal mechanisms available for achieving this coverage: (i) hardware redundancy, (ii) hardware-based Built-In Self-Test (BIST) mechanisms, and (iii) software-based BIST mechanisms. **While hardware redundancy is an effective method, it is generally eschewed in non-mission-critical modules due to the substantial area overhead it incurs. Hardware-based BIST mechanisms are frequently utilized; however, their implementation may not be feasible under certain conditions [8], [9].** For instance, a given module may not have been originally designed with BIST capabilities. Furthermore, such mechanisms necessitate that the device transitions into a scan/Automatic Test Pattern Generation (ATPG) mode.

The transition to ATPG mode requires the preservation of the complete Intellectual Property (IP) or module context prior to testing, along with its restoration afterward, to ensure seamless execution of the application. **This procedure may prove impractical within the constraints of the designated testing time window.** This invasive reconfiguration of the system into a non-operational state during testing not only disrupts system functionality but also risks overtesting, resulting in downtime and inefficiencies. Such interruptions render these techniques unsuitable, particularly in safety-critical systems

*Corresponding author: Sanjay Das (Email: sanjay.das@utdallas.edu)

This work is supported by the Semiconductor Research Corporation (SRC GRC Task: 3192.001) and NSF award # 2343652.

like automotive applications, where continuous operation is essential. **Therefore, non-scan-based methods, such as software-based self-test (SBST) mechanisms, offer a more flexible and practical approach to achieving the essential fault coverage required in contemporary systems [10].**

For in-field testing, where halting system operations is impractical, the invasive nature of design-for-testability (DfT) techniques presents a significant challenge. In contrast, SBST approaches, such as self-test libraries (STLs), provide a more efficient and less invasive solution [11], [12]. STLs execute functional programs to detect faults without altering the design or interrupting system operations. These libraries can be run at full operational speed, enabling fault detection during normal operation without any disruption. Moreover, STLs can divide the testing process into small segments during idle periods, making them ideal for in-field applications. This method not only prevents overtesting but also maintains system integrity and supports long-term functional safety, in line with stringent standards like ISO 26262. Therefore, the use of STLs allows for continuous and non-intrusive fault detection, ensuring the system's reliability and safety throughout its operational lifecycle without compromising performance or introducing inefficiencies. **Commercial STLs are proprietary, device-specific, and typically handcrafted for each target design. Therefore, it becomes incumbent to automate the STL generation based on the functional test patterns in order to optimize SBST overhead.** ML-based pattern generation can aid in this automated STL-based testing by increasing the fault coverage. This can reduce the memory requirement to periodically run tests on the device when it is in use.

In this work, we propose a novel framework for automated STL generation to ensure FuSa in mission-critical applications by enabling in-field testing with minimal operational disruption. Our framework introduces an innovative ML-based approach for generating functional test patterns without relying on scan-based test mode assertions, thereby avoiding operational disruptions. It leverages advanced machine learning techniques, such as Variational Autoencoders (VAE) and Reinforcement Learning (RL), to optimize test pattern generation. **Since this is the first automated STL generation technique, our approach is evaluated against random pattern generation methods.** Our key contributions in this work include:

- We introduce a novel, automated framework for STL generation that ensures FuSa in mission-critical applications by enabling in-field testing with minimal operational disruption.
- We propose an ML-driven framework to generate functional test patterns without the need for traditional scan-based testing methods.
- The proposed ML framework integrates VAE with RL, using VAE to initialize the RL-based pattern optimization process, ensuring effective exploration of the solution space.
- The proposed solution delivers high operational fault coverage, with up to 57.57% improvement over random pattern-based testing, while avoiding the pitfalls of test mode assertion and system downtime associated with ATPG-based techniques.

II. BACKGROUND

In this section, we provide the essential background necessary to elucidate our proposed approach for machine learning-based test pattern generation and the development of STLs.

A. Self-Test Library

The Self-Test Library (STL) in VLSI testing is a suite of tools and methodologies designed to facilitate efficient on-chip testing

of integrated circuits (ICs) [12]. It includes techniques like Built-In Self-Test (BIST) and non-scan-based testing to detect faults and ensure circuit functionality without external testing equipment. The STL allows for the embedded test circuitry to perform in-situ tests, which is particularly beneficial for detecting manufacturing defects and operational faults in complex VLSI circuits, reducing reliance on external testers and equipment.

Compared to traditional methods like BIST and ATPG, STLs offer advantages in terms of flexibility and scalability. BIST introduces test pattern generators and output response analyzers within the chip, providing autonomous testing but often at the cost of significant area, power, and timing overheads. ATPG, which typically uses scan chains to shift test patterns, also adds overhead in terms of area and performance but can be limited by the length and complexity of the scan chains. In contrast, STLs, especially those leveraging non-scan-based techniques, provide a more scalable solution that can adapt to both sequential and combinational circuits, reducing external resource dependence. For non-scan-based testing, techniques such as Linear Feedback Shift Registers (LFSRs) and pseudorandom pattern Generators (PRPGs) are employed to generate high fault coverage test patterns [13], [14]. These methods are effective in achieving high coverage with minimal area and test time overheads, as LFSRs and PRPGs efficiently produce large sets of pseudo-random patterns that can detect a wide range of faults. While these techniques do not completely eliminate the need for scan chains in some designs, they help significantly reduce the overhead associated with traditional scan-based testing.

Thus, STLs provide a robust mechanism to enhance the testability and reliability of VLSI circuits by combining the efficiency of on-chip testing with reduced dependence on external testing resources. This makes it a preferred choice for modern IC designs, particularly in domains requiring high reliability, such as automotive and aerospace systems [15], [16]. The manual and proprietary nature of commercial STLs, tailored specifically for individual designs, creates significant challenges in scalability and standardization. To overcome these issues, this work, for the first time, introduces an automated STL generation framework powered by machine learning, aiming to improve FuSa in mission-critical systems.

B. Variational Autoencoder

A Variational Autoencoder (VAE) is a type of generative model used for unsupervised learning, built on the foundation of traditional autoencoders [17]. To understand VAEs, it is essential to first grasp the concept of a standard autoencoder. An autoencoder is a neural network designed to learn a compressed, lower-dimensional representation (encoding) of input data, such as images or text, and then reconstruct the input from this representation. The goal is to learn useful features or patterns from the data by minimizing the difference between the original input and its reconstruction. This process is driven by a loss function (to represent the said difference), usually the mean squared error or cross-entropy.

The VAE extends this idea by introducing a probabilistic interpretation of the latent space (the space where the compressed data representation resides). Instead of learning fixed values for the compressed representation, the VAE assumes that the data is generated from an underlying probability distribution. In this way, the encoder network learns to output the parameters of this distribution (typically a mean and variance) rather than a single deterministic point. A key innovation in VAEs is the reparameterization trick, which allows for back-propagation through the stochastic sampling of the latent variables by expressing the randomness in a differentiable way. During training,

the VAE attempts to reconstruct the input data accurately and ensure that the learned latent space follows a known probability distribution (often a normal distribution). This is achieved by optimizing a loss function that includes two components: the reconstruction loss, which measures how well the VAE can recreate the input data, and a regularization term called the Kullback-Leibler divergence [18]. This term ensures that the learned distribution is close to the prior distribution, which allows for efficient sampling of new, unseen data points from the model after training. This combination of learning a probabilistic latent space and reconstruction capability makes VAEs a powerful tool for generating new data and discovering underlying patterns in complex datasets, which we exploit for our proposed test pattern generation procedure.

C. Reinforcement Learning

Reinforcement Learning (RL) is a specialized area of machine learning that focuses on enabling agents to learn optimal behaviors through interactions with their environment [19]. Distinct from supervised learning, where models learn from labeled datasets, RL requires agents to make decisions within uncertain and dynamic settings to maximize cumulative rewards over time. The core components of an RL framework include the agent, the environment, states, actions, and rewards. In this paradigm, the agent represents the learner or decision-maker, while the environment encompasses all external factors influencing the agent. States reflect the current condition of the environment, and actions are the choices available to the agent that may alter this condition. Upon executing an action, the agent receives a reward, quantifying the immediate benefit derived from that action. The primary objective is to derive a policy—a mapping from states to actions—that maximizes expected cumulative rewards.

Reinforcement Learning algorithms typically utilize trial-and-error methods to explore the environment, balancing exploration (trying new actions) with exploitation (selecting known advantageous actions). Common strategies to achieve this balance include the epsilon-greedy approach and Upper Confidence Bound (UCB) algorithms. Notable RL methodologies encompass model-free techniques, such as Q-learning and Policy Gradient methods, and model-based approaches, which involve constructing a model of the environment's dynamics. Q-learning, a value-based method, estimates the expected utility of actions in given states, while Policy Gradient methods directly optimize the policy, accommodating complex action spaces.

Reinforcement Learning has demonstrated effectiveness across diverse domains, including robotics, game playing, and resource management, due to its capacity for handling intricate decision-making tasks and adapting to dynamic environments. Therefore, we translate our STL generation process into an RL-based optimization for enhancing FuSa analysis.

III. METHODOLOGY

In this section, we present a comprehensive methodology for generating STLs comprising high-quality functional test patterns aimed at enhancing fault coverage in hardware systems. Our approach integrates advanced machine learning techniques, including VAE and RL, to optimize test pattern generation as illustrated in Figure 2. Moreover, this framework incorporates an STL generation process that enables non-intrusive, in-field testing without halting system operations.

A. VAE-Based Learning for Functional Pattern Generation

In this section, we describe the process of developing and using a VAE to generate new test patterns for high fault coverage in E/E

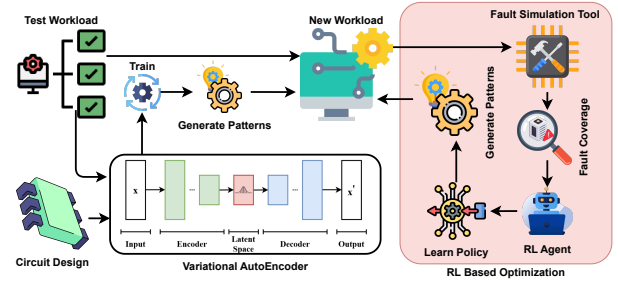


Fig. 2. Overview of the proposed test pattern generation framework.

systems. The proposed approach leverages both the structure of the design and the existing set of functional test patterns. The primary goal is to learn the underlying distribution of the test patterns and generate new patterns that effectively target critical regions of the design.

We start by preparing the input data, which includes a set of test patterns P and the design netlist graph G . The netlist graph is first encoded into a feature representation, G_{features} , using a graph neural network, where the node features represent the circuit components and the edges capture the connectivity. This encoding allows us to incorporate the structural information of the design into the learning process. Once G_{features} is obtained, we concatenate it with each test pattern P from the pattern set. This combined representation $[P, G_{\text{features}}]$ is then passed through the encoder of the VAE, which transforms it into a latent vector z by approximating the posterior distribution using a multivariate Gaussian:

$$z \sim q_{\phi}(z|[P, G_{\text{features}}]) = \mathcal{N}(\mu_{\phi}([P, G_{\text{features}}]), \sigma_{\phi}([P, G_{\text{features}}])) \quad (1)$$

Here, μ_{ϕ} and σ_{ϕ} are the mean and standard deviation vectors learned by the encoder, parameterized by ϕ .

Next, we sample the latent vector z from this distribution and pass it through the decoder to generate a reconstructed test pattern $\hat{P}$:

$$\hat{P} = \text{Decoder}_{\phi}(z) \quad (2)$$

This process allows the VAE to reconstruct patterns that are similar to the original ones but with variability introduced by the latent space, thus encouraging the generation of novel test patterns.

To ensure that the generated patterns are meaningful, we define a reconstruction loss L_{recon} , which measures the difference between the original patterns P and the reconstructed patterns $\hat{P}$. This is typically computed as the mean squared error:

$$L_{\text{recon}} = \frac{1}{B} \sum_{i=1}^B \|P_i - \hat{P}_i\|^2 \quad (3)$$

where B is the batch size of patterns processed during training. In addition to the reconstruction loss, we impose a Kullback-Leibler (KL) divergence term L_{KL} to regularize the latent space. The KL divergence ensures that the learned distribution $q_{\phi}(z|P)$ remains close to a standard normal distribution $p(z)$:

$$L_{\text{KL}} = D_{\text{KL}}(q_{\phi}(z|P) \| p(z)) \quad (4)$$

The total loss L that we minimize during training is a combination of the reconstruction loss and the KL divergence, weighted by a hyper-parameter λ :

$$L = L_{\text{recon}} + \lambda L_{\text{KL}} \quad (5)$$

This regularization not only prevents overfitting but also ensures that the latent space is smooth and can generate a wide variety of test

Algorithm 1 VAE Learning from Pattern Set

Input: Pattern set $\mathcal{P}$, design netlist graph $\mathcal{G}$, learning rate α , number of epochs E , batch size B

- 1: Initialize VAE model VAE_ϕ with parameters ϕ
- 2: Initialize optimizer for VAE_ϕ
- 3: **for** each epoch $e = 1$ to E **do**
- 4: Shuffle the pattern set $\mathcal{P}$
- 5: **for** each mini-batch of patterns P_b from $\mathcal{P}$ **do**
- 6: Encode $\mathcal{G}$ into a graph feature representation G_{features}
- 7: **for** each pattern $P \in P_b$ **do**
- 8: Concatenate pattern P with G_{features}
- 9: Pass data through VAE encoder to obtain latent vector z
- 10: Decode latent vector z to reconstruct pattern $\hat{P}$
- 11: **end for**
- 12: Compute reconstruction loss $\mathcal{L}_{\text{recon}}$ between P_b and $\hat{P}_b$
- 13: Compute KL divergence loss $\mathcal{L}_{\text{KL}}$
- 14: Total loss $\mathcal{L} = \mathcal{L}_{\text{recon}} + \lambda \mathcal{L}_{\text{KL}}$
- 15: Back-propagate and update VAE parameters ϕ
- 16: **end for**
- 17: **end for**
- 18: **Output:** Trained VAE model VAE_ϕ

patterns. By training the VAE on the existing pattern set P , we effectively learn a latent space that captures the essential features of the design and its corresponding test patterns. Once training is complete, we can generate new test patterns by sampling from the latent space $z \sim \mathcal{N}(0, 1)$ and decoding them into test patterns using the decoder ($\hat{P}_{\text{new}} = \text{Decoder}_\phi(z)$).

The VAE-based learning and pattern generation is outlined in Algorithm 1, that begins by initializing the VAE model with parameters ϕ and setting up an optimizer for the VAE model (lines 1-2). Then, for each epoch from 1 to E , the pattern set P is shuffled to ensure randomness in the training process (lines 3-4). Next, for each mini-batch of patterns P_b drawn from the shuffled pattern set, the design netlist graph G is encoded into a graph feature representation G_{features} (lines 5-6). Subsequently, each pattern P in the mini-batch is concatenated with G_{features} to create the input data for the VAE encoder (lines 7-8). The concatenated data is then passed through the VAE encoder to generate a latent vector z , which is sampled from the encoder (line 9). The latent vector is decoded to reconstruct the pattern $\hat{P}$ (line 10). Subsequently, the reconstruction loss $\mathcal{L}_{\text{recon}}$ is calculated between the original patterns and their reconstructed versions (line 12), followed by the computation of the KL divergence loss $\mathcal{L}_{\text{KL}}$ for latent space regularization (line 13). The total loss $\mathcal{L}$ is then computed as the sum of the reconstruction loss and the regularization loss (line 14). Finally, the VAE parameters ϕ are updated using back-propagation through the optimizer (line 15). The process repeats over the mini-batches for each epoch until the VAE model is fully trained, at which point the trained VAE model VAE_ϕ is output (lines 16-18).

These newly generated patterns are tailored to the specific characteristics of the design and provide enhanced test coverage by targeting regions of the design that may not have been fully excited by the original patterns.

B. RL-based Functional Test Pattern Optimization

In this section, we elaborate on the integration of a trained VAE with a RL framework to efficiently generate high-quality functional test patterns. The VAE, which learns the underlying distribution of existing test patterns, is used to sample potential new patterns, while

Algorithm 2 Reinforcement Learning for Pattern Generation

Input: Design netlist $\mathcal{D}$, fault analysis tool $\mathcal{F}$, pre-trained VAE model VAE_ϕ

- 1: Initialize policy model π_θ with random weights θ
- 2: Initialize best fault coverage $C_{\text{best}} \leftarrow 0$
- 3: Initialize reward $r \leftarrow 0$
- 4: **for** each episode $t = 1$ to T **do**
- 5: Sample $z \sim \mathcal{N}(0, I)$ from the VAE's latent space
- 6: Decode z to generate an initial pattern $P_0 = \text{VAE}_\phi(z)$
- 7: Apply pattern P_0 compute initial fault coverage C_0
- 8: **for** each step $k = 1$ to K **do**
- 9: Perturb P_{k-1} using π_θ to generate P_k
- 10: Apply pattern P_k to compute new fault coverage C_k
- 11: Calculate reward: $r_k = C_k - C_{k-1}$
- 12: Update policy model π_θ using the reward r_k
- 13: **if** $C_k > C_{\text{best}}$ **then**
- 14: Update $C_{\text{best}} \leftarrow C_k$
- 15: **end if**
- 16: **if** reward $r_k \leq 0$ (no improvement) **then**
- 17: Break the loop and stop perturbing
- 18: **end if**
- 19: **end for**
- 20: **end for**
- 21: **Output:** The pattern P_{best} that achieves the highest coverage C_{best}

the RL agent optimizes the selection and refinement of these patterns to maximize fault coverage. This combined approach leverages the generative capabilities of the VAE and the optimization power of RL to dynamically adapt the pattern generation process, improving overall testing efficiency.

The VAE model samples patterns from a latent space, represented by $z \sim \mathcal{N}(0, 1)$, and decodes this latent vector into test patterns $\hat{P} = \text{Decoder}(z)$. These generated patterns are then evaluated by the RL agent, which learns a policy $\pi(a_t|s_t)$ to decide how to use the patterns in the context of the design. Here, s_t represents the system state at time step t , which encodes critical design information, such as circuit netlist features G_{features} and current fault coverage metrics. The agent's action a_t , chosen from a predefined action space, can involve sampling a new test pattern from the VAE, modifying an existing one, or reapplying a previously used pattern, depending on the current state.

The reward function for the RL agent is tied to improvements in fault coverage. At each time step, the reward r_t is calculated as the increase in coverage from the previous step:

$$r_t = \text{Coverage}(t) - \text{Coverage}(t-1) \quad (6)$$

This reward structure encourages the agent to prioritize actions that lead to significant fault coverage improvements, penalizing ineffective or redundant actions. The RL agent's goal is to maximize the cumulative discounted reward R_t , defined as:

$$R_t = \mathbb{E} \left[\sum_{k=t}^T \gamma^{k-t} r_k \right] \quad (7)$$

where γ is the discount factor. Policy gradient methods, such as Proximal Policy Optimization (PPO), are employed to iteratively refine the agent's policy $\pi(a_t|s_t)$, ensuring that the agent learns to select actions that yield optimal test coverage.

The RL-based test pattern optimization is detailed in the Algorithm 2, which starts by initializing the policy model π_θ with random

weights θ , setting the best fault coverage C_{best} to 0, and initializing the reward r to 0 (lines 1-3). For each episode from 1 to T , a latent vector z is sampled from the VAE's latent space, where z follows a normal distribution $N(0, I)$ (lines 4-5). This latent vector is decoded by the pre-trained VAE model VAE_ϕ to generate an initial test pattern P_0 (line 6). The generated pattern P_0 is applied to the design netlist D , and its initial fault coverage C_0 is computed using the fault analysis tool F (line 7). Next, for each step from 1 to K , the current pattern P_{k-1} is perturbed by the policy model π_θ to generate a new pattern P_k (line 9). This new pattern P_k is applied to the design netlist D , and the updated fault coverage C_k is calculated using the fault analysis tool F (line 10). The reward r_k is then computed as the difference between the new fault coverage C_k and the previous fault coverage C_{k-1} (line 11). The policy model π_θ is updated based on the reward r_k (line 12). If the new fault coverage C_k surpasses the best fault coverage C_{best} , the best fault coverage C_{best} is updated to C_k (lines 13-14). However, if the reward r_k is less than or equal to 0, meaning no improvement in fault coverage, the loop is terminated, and further perturbation stops (lines 16-17). This process repeats for each episode until all episodes are completed (lines 19-20). Finally, the algorithm outputs the pattern P_{best} that achieved the highest fault coverage C_{best} (line 21).

C. Self-Test Library (STL) Generation for In-Field Testing

In this section, we outline the methodology for generating a STL from functional test patterns to facilitate efficient in-field testing of VLSI designs. The STL enables continuous fault detection and real-time fault coverage evaluation without halting system operations, as illustrated in the Figure 3.

The process begins with the utilization of functional test patterns, $P = \{p_1, p_2, \dots, p_N\}$, generated by the RL-VAE approach. Each pattern p_i consists of an input-output mapping derived from the design's expected behavior, $f_{\text{design}} : I \rightarrow O$, where I represents the set of input signals and O the corresponding output signals.

These patterns are transformed into machine-executable instructions by translating the RTL description of the design into a higher-level programming language, typically C or C++. The transformation can be expressed as:

$$C_{\text{design}} = \mathcal{T}(V_{\text{design}}) \quad (8)$$

where $\mathcal{T}$ represents the transformation function that converts the Verilog representation V_{design} into its C/C++ equivalent C_{design} .

Once transformed, the STL generation process automates the creation of testbenches. For each pattern p_i , the corresponding input vector I_i is assigned to the input signals of the testbench, which drives the evaluation of the C++ design model. The process involves repeatedly evaluating the system's response to each input vector:

$$O_i^{\text{actual}} = f_{\text{STL}}(I_i) \quad (9)$$

where f_{STL} represents the behavior of the design under test as executed by the generated STL. The actual output O_i^{actual} is compared against the expected output O_i^{expected} to detect discrepancies or faults:

$$\Delta O_i = O_i^{\text{actual}} - O_i^{\text{expected}} \quad (10)$$

If $\Delta O_i \neq 0$, the test reveals a fault.

The STL is integrated into the system's memory, enabling non-intrusive execution during idle cycles or specific test periods. This reduces the performance overhead by running tests concurrently with normal operations.

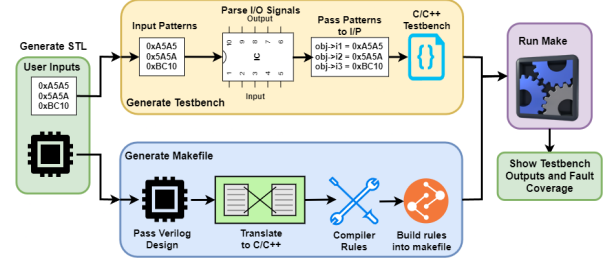


Fig. 3. Flow of processes to generate STL.

IV. EXPERIMENTAL EVALUATION

A. Experimental Setup

The proposed framework is evaluated using four distinct designs: a Synchronous Dynamic Random Access Memory (SDRAM) Controller, the OpenCores Memory Controller Main Timing (OCMCMT) module, the OpenRISC 1200 instruction fetch (ORIF) module, and a Single Port Synchronous RAM (ORSPRAM) module. These designs represent a variety of circuitry and critical functionalities, providing a comprehensive basis for evaluating the framework's effectiveness.

To construct the initial dataset, each design's register-transfer level (RTL) representation is synthesized into a gate-level netlist using Synopsys Design Vision. Fault injection experiments are then conducted using Cadence Xcelium [20], which simulates the circuit and injects faults across different input vectors (workloads). Fault detection reports are generated, detailing fault criticalities and coverage metrics. These reports are compiled into a dataset that is used to train a VAE, which learns the latent distribution of test patterns and generates new synthetic patterns. Upon training, the VAE is used in the RL-based framework to generate high-quality test patterns aimed at maximizing fault coverage. For generating the STL, we employ Verilator [21], an open-source tool known for its efficiency in converting high-level hardware designs into cycle-accurate simulations, thus facilitating the rapid verification of the design during STL development.

B. Performance Metrics

Model performance is assessed using fault coverage, representing the percentage of detected faults by the applied functional test patterns. We also use the number of patterns needed as another metric to showcase the efficiency of the pattern-generation process. A higher fault coverage indicates better model performance, while a lower number of required patterns shows the efficiency of the pattern generation approach.

C. STL Generation

The process of generating an STL involves the use of a script that takes a Verilog RTL design and an input pattern set as its primary inputs, producing four essential output files. To ensure compatibility, these input files must adhere to a specific format. Below, we detail the STL generation procedure using the ORIF design as an example.

In this process, great care is taken to avoid conflicts with the module's port names. Each signal in the generated files is distinctly separated, and comments are carefully structured to avoid the use of reserved keywords such as *input* and *output*, minimizing the risk of errors or ambiguities during module port parsing. This precaution ensures a smooth and error-free generation process.

The four files produced by the script are the Makefile, Makefile_obj, inputs.vc, and a C++ testbench. The Makefile is responsible for compiling the C++ executable using Verilator, that converts RTL designs into C++ models. It handles

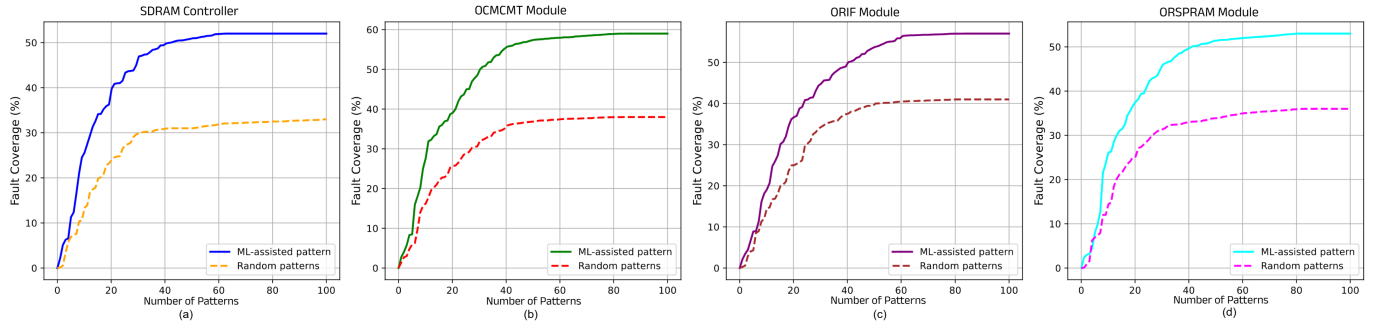


Fig. 4. Fault coverage analysis across various designs: (a) SDRAM controller, (b) OCMCMT module, (c) ORIF module and (d) SPRAM module.

Snippet of the generated testbench

```
void test_orl200_if() {
    Vorl200_if *tb = new Vorl200_if;
    // Initialize all input signals to 0
    tb->clk = 0; tb->rst = 0;
    tb->icpu_dat_i = 0;
    tb->icpu_adr_i = 0;
    ...
    // Apply input patterns
    // Pattern 1
    tb->rst = 1;
    tb->icpu_dat_i = 0xAAAA;
    tb->icpu_adr_i = 0x1000;
    ...
    tb->clk = !tb->clk;
    tb->eval();
    // Simulate time progression
    Verilated::timeInc(1);
    //Add more test patterns
    delete tb;
}
```

the setup of basic compiler flags and initiates the Verilation process, which translates RTL signals into C++ object representations. The `Makefile_obj` file manages the building and linking of the Verilated design objects referenced during the compilation process. It specifies the necessary compilation flags, linking rules, and execution procedures for the C++ hardware executables. Additionally, the script generates an `inputs.vc` file, which imports the required libraries and directories. This file is modular, allowing for easy modification and management of project imports, ensuring that all dependencies remain organized.

Finally, the input pattern set is used to generate a C++ testbench, which is employed to simulate the RTL design and conduct fault coverage analysis. Paired with the `Makefile` and `Makefile_obj`, the testbench enables the complete execution of the STL, simulating the functionality of the design based on the provided input test patterns. Through this simulation process, the STL aids in verifying the design's fault coverage and ensuring its operational integrity. We will open source the framework upon acceptance.

D. Fault Coverage Results

In our evaluation, we examined four distinct hardware designs to assess the efficacy of our fault coverage improvement method compared to random patterns for a specific number of patterns, which is set to 100 for our analysis, to reduce the overhead (often associated with ATPG-based techniques as discussed in Section I). Table I compares fault coverage achieved by two pattern generation methods: ML-assisted and random pattern generation. It includes design details, the number of gates and flip-flops, fault coverage for each method, and the improvement provided by the ML-assisted approach. The

TABLE I
COMPARISON OF FAULT COVERAGE ANALYSIS.

Design	Gates	Flip-flops	Maximum fault coverage (%) with 100 patterns		# ML functional patterns to match random coverage
			Random	ML	
SDRAM Controller	312	91	33%	52%	15
OCMCMT Module	873	188	38%	59%	18
ORIF Module	238	67	41%	57%	25
ORSPRAM Module	625	132	36%	53%	19

results emphasize the superior performance of ML-assisted pattern generation in enhancing fault detection.

For the SDRAM Controller, random patterns achieved 33% fault coverage with 100 test patterns. In contrast, ML-assisted patterns improved fault coverage to 52%, as shown in Figure 4(a). Moreover, using only 15 ML-assisted patterns achieved the same 33% fault coverage as 100 random patterns, demonstrating an 85% efficiency improvement.

The OCMCMT module showed 38% fault coverage with random patterns, while ML-assisted patterns increased coverage to 59%, as illustrated in Figure 4(b). Remarkably, 18 ML-assisted patterns matched the 38% coverage of random patterns, yielding an 82% efficiency gain while maintaining baseline performance.

For the ORIF module, random patterns achieved 41% fault coverage, which improved to 57% with ML-assisted patterns, as shown in Figure 4(c). Similarly, the SPRAM module exhibited 36% fault coverage with random patterns, while ML-assisted patterns raised coverage to 53%, as depicted in Figure 4(d).

The comparative analysis presented in Table I demonstrates that ML-assisted pattern generation significantly improves fault coverage across all evaluated designs. These improvements include up to a 57.57% increase in fault coverage compared to random patterns and an efficiency enhancement of up to 85%. These results underscore the effectiveness of ML-assisted methodologies in optimizing fault detection capabilities in integrated circuit designs.

V. CONCLUSION

The proposed ML-driven STL generation framework addresses the limitations of traditional fault detection techniques by enabling seamless, non-intrusive in-field testing for safety-critical hardware systems. Unlike commercial STLs, our approach automates the STL generation, enabling less manual effort. By leveraging reinforcement learning, the STL generates design-specific test patterns that achieve up to a 57.57% improvement in fault coverage and up to 85% testing efficiency compared to random pattern-based methods. This approach ensures uninterrupted system operation during fault detection, making it highly suitable for mission-critical domains such as automotive, aerospace, and medical devices. These results demonstrate the framework's potential to significantly enhance functional safety in increasingly complex hardware systems.

REFERENCES

- [1] R. Palin, D. Ward, I. Habli, and R. Rivett, "Iso 26262 safety cases: Compliance and assurance," 2011.
- [2] A. Chaudhuri *et al.*, "Fault-criticality assessment for ai accelerators using graph convolutional networks," in *2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2021, pp. 1596–1599.
- [3] M. Reddy, B. Satyanarayana, M. Ravi, P. Krishnaiah, C. Dileep, and B. Annapoorna, "Machine learning-based fault tolerance techniques for vlsi circuit design," in *International Conference on Data Science, Machine Learning and Applications*. Springer, 2023, pp. 1359–1369.
- [4] Y. Wang, J. Xiao, Z. Wei, Y. Zheng, K.-T. Tang, and C. H. Chang, "Security and functional safety for ai in embedded automotive system—a tutorial," *IEEE Transactions on Circuits and Systems II: Express Briefs*, 2023.
- [5] N. Akter, M. R. Siddiquee, M. Shur, and N. Pala, "Ai-powered terahertz vlsi testing technology for ensuring hardware security and reliability," *IEEE Access*, vol. 9, pp. 64 499–64 509, 2021.
- [6] S. Das, S. Kundu, P. Madhusoodhanan, P. Viswanathan Pillai, R. Parekhji, A. Raha, S. Banerjee, S. Natarajan, and K. Basu, "Graph learning-based fault criticality analysis for enhancing functional safety of e/e systems," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, 2024, pp. 1–6.
- [7] R. Bell, "Introduction to iec 61508," in *Acm international conference proceeding series*, vol. 162. Citeseer, 2006, pp. 3–12.
- [8] S. Shibahara, "Functional safety soc for autonomous driving," in *2018 IEEE Custom Integrated Circuits Conference (CICC)*, 2018, pp. 1–8.
- [9] C. Eychenne and Y. Zorian, "An effective functional safety infrastructure for system-on-chips," in *2017 IEEE 23rd International Symposium on On-Line Testing and Robust System Design (IOLTS)*. IEEE, 2017, pp. 63–66.
- [10] A. Dutta, M. Shah, G. Swathi, and R. A. Parekhji, "Design techniques and tradeoffs in implementing non-destructive field test using logic bist self-test," in *2009 15th IEEE International On-Line Testing Symposium*. IEEE, 2009, pp. 237–242.
- [11] L. Anghel, R. Cantoro, R. Masante, M. Portolan, S. Sartoni, and M. S. Reorda, "Self-test library generation for in-field test of path delay faults," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 11, pp. 4246–4259, 2023.
- [12] R. Cantoro, P. Girard, R. Masante, S. Sartoni, M. S. Reorda, and A. Virazel, "Self-test libraries analysis for pipelined processors transition fault coverage improvement," in *2021 IEEE 27th International Symposium on On-Line Testing and Robust System Design (IOLTS)*. IEEE, 2021, pp. 1–4.
- [13] K. Devika and R. Bhakthavatchalu, "Design of reconfigurable lfsr for vlsi ic testing in asic and fpga," in *2017 international conference on communication and signal processing (ICCSP)*. IEEE, 2017, pp. 0928–0932.
- [14] T. Garbolino, "A new, fast pseudo-random pattern generator for advanced logic built-in self-test structures," *Applied Sciences*, vol. 11, no. 20, p. 9476, 2021.
- [15] P. Bernardi, R. Cantoro, A. Floridaia, D. Piumatti, C. Pogonea, A. Ruospo, E. Sánchez, S. De Luca, and A. Sansonetti, "Non-intrusive self-test library for automotive critical applications: Constraints and solutions," in *2019 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2019, pp. 920–923.
- [16] F. Pratas, T. Dedes, A. Webber, M. Bemanian, and I. Yarom, "Measuring the effectiveness of iso26262 compliant self test library," in *2018 19th International Symposium on Quality Electronic Design (ISQED)*. IEEE, 2018, pp. 156–161.
- [17] L. Pinheiro Cinelli, M. Araújo Marins, E. A. Barros da Silva, and S. Lima Netto, "Variational autoencoder," in *Variational Methods for Machine Learning with Applications to Deep Networks*. Springer, 2021, pp. 111–149.
- [18] T. Kim, J. Oh, N. Kim, S. Cho, and S.-Y. Yun, "Comparing kullback-leibler divergence and mean squared error loss in knowledge distillation," *arXiv preprint arXiv:2105.08919*, 2021.
- [19] L. P. Kaelbling, M. L. Littman, and A. W. Moore, "Reinforcement learning: A survey," *Journal of artificial intelligence research*, vol. 4, pp. 237–285, 1996.
- [20] Cadence, "Xcelium fault simulator," 2024, https://www.cadence.com/en_US/home/training/all-courses/86246.html, Last accessed on 2024-09-10.
- [21] W. Snyder, "Verilator 4.0: open simulation goes multithreaded," in *Open Source Digital Design Conference (ORConf)*, 2018.